



```
// concatenar
s += "abc";

// agregar caracter
s.push_back('x');

// quitar último
s.pop_back();

// substring
s.substr(pos, len)

// Ejemplo:

string s = "abcdef";

s.substr(2,3) -> "cde"

// Complejidad
//O(len)

// encontrar substring
s.find("abc")

// retorna
// posición o string::npos

// recorrer string (forma correcta)
// rápido
for(char c : s)

// con índice
for(int i = 0; i < s.size(); i++)

// leer strings (ICPC)
// palabra
string s;
cin >> s;

// línea completa
getline(cin, s);

// comparar strings
if (a == b)
//lexicográfico
if (a < b)
```



```
// invertir string
reverse(s.begin(), s.end());

// ordenar string
sort(s.begin(), s.end());

// contar caracteres
vector<int> freq(26);

for(char c : s)
    freq[c - 'a']++;

// convertir char ↔ int
// char → int
int x = c - '0';
// int → char
char c = x + '0';

// lowercase / uppercase
tolower(c)
toupper(c)
```

```

// eliminar caracteres
// Ejemplo: borrar todos los a
s.erase(remove(s.begin(), s.end(), 'a'), s.end());

// split string (muy común)
// Ejemplo:
// "a,b,c"
vector<string> split(string s, char delim) {
    vector<string> res;
    string cur;

    for(char c : s){
        if(c == delim){
            res.push_back(cur);
            cur.clear();
        }
        else cur += c;
    }

    res.push_back(cur);
    return res;
}

// verificar palíndromo
bool pal(string s){
    int l=0,r=s.size()-1;

    while(l<r){
        if(s[l]!=s[r]) return false;
        l++; r--;
    }

    return true;
}

// contar substrings
// Ejemplo:
abc
// substrings
a
b
c
ab
bc
abc

// Código:

for(int i=0;i<n;i++)
    for(int j=i;j<n;j++)
        cout << s.substr(i,j-i+1);

// patrones IMPORTANTES ICPC
int l=0,r=0;

while(r<n){
    while(condicion)
        l++;

    r++;
}

// Se usa para:
// substrings
// ventanas

```



```
is_palindrome = lambda s: s == s[::-1]
print(is_palindrome("madam")) # true
```



```
# remove duplicates
nums = [1, 2, 2, 3, 3, 3]
print(list(set(nums)))
```



```
// Contar todas las letras.  
int freq[256]={0};  
  
for(char c:s)  
    freq[c]++;  
  
// Sirve para ASCII completo.  
  
// convertir string ↔ número  
    // string → int  
stoi(s)  
    // string → long long  
stoll(s)  
    // int → string  
to_string(x)  
  
// borrar substring  
s.erase(pos, len);  
  
// insertar substring  
s.insert(pos, "abc");  
  
// Evita concatenar así:  
s = s + c  
// Mejor:  
s.push_back(c)
```